

Nome do autor: Lucas Ruan Fidélis Ferreira

Título do trabalho: Triagem Eficiente de Alertas em Centros de Operações de Segurança por meio de um Pipeline Auto-Hospedado em Duas Etapas com LLMs

Nome do orientador: Felipe A. P. de Figueiredo e Evandro C. Vilas Boas

Nome da instituição de vínculo: Instituto Nacional de Telecomunicações - Inatel

Endereço: Av. João de Camargo, 510, Santa Rita do Sapucaí - MG, CEP 37536-001

Telefone: +55 (35) 3471-9200

E-mail: inatel@inatel.br

Nome da instituição de onde se desenvolveu a pesquisa: Instituto Nacional de Telecomunicações - Inatel

Endereço: Av. João de Camargo, 510, Santa Rita do Sapucaí - MG, CEP 37536-001

Telefone: +55 (35) 3471-9200

E-mail: inatel@inatel.br

1 Resumo

A crescente complexidade das ameaças cibernéticas e o alto volume de alertas em Centros de Operações de Segurança (SOCs) têm ampliado a sobrecarga dos analistas e reduzido a eficiência do processo de triagem. Este trabalho apresenta um pipeline auto-hospedado em duas etapas, voltado para a triagem inteligente de alertas, que integra modelos de linguagem de grande porte (LLMs) leves e robustos sem depender de serviços externos. Na primeira etapa, modelos leves realizam a filtragem dos logs associados aos alertas, removendo informações irrelevantes e reduzindo o custo computacional da análise posterior. Na segunda etapa, modelos mais robustos classificam os alertas por meio de estratégias de *ensemble*, incluindo votação por maioria, votação ponderada e seleção dinâmica, com o objetivo de aumentar a precisão e a consistência das decisões. Os experimentos foram conduzidos em um ambiente controlado de Windows com simulações de ataques utilizando o Atomic Red Team, integrado a uma plataforma SIEM, o que permitiu gerar uma base de dados representativa e alinhada ao framework MITRE ATT&CK. Os resultados mostraram que a abordagem proposta alcançou mediana de F1-score de 0,90 e *recall* de 0,95, superando tanto modelos individuais quanto algoritmos tradicionais de aprendizado de máquina. Como aplicação prática, a solução contribui para reduzir a fadiga de alertas, preservar a privacidade dos dados sensíveis de segurança e tornar viável o uso de inteligência artificial em operações reais de SOC com infraestrutura própria.

Palavras-chave: Inteligência Artificial, Cibersegurança, SOC, LLM, Machine Learning.

2 Introdução

A aplicação de Inteligência Artificial (IA) à cibersegurança tem se consolidado como uma alternativa relevante para lidar com o crescimento da complexidade das ameaças digitais. O impacto econômico dos incidentes cibernéticos é expressivo: o custo global do cibercrime foi estimado em USD 7,08 trilhões em 2022, com projeção de alcançar USD 13,82 trilhões até 2028 [1]. Nesse contexto, soluções baseadas em IA oferecem vantagens importantes, como a análise de grandes volumes de dados em tempo real, a redução da carga cognitiva dos analistas e a identificação de padrões sutis que podem passar despercebidos por métodos tradicionais [2]. Além disso, a IA permite automatizar tarefas repetitivas, como triagem inicial de alertas e detecção de anomalias, aumentando a eficiência operacional e reduzindo o tempo de resposta a incidentes [3]. Outro ponto relevante é a capacidade de operar continuamente, sem fadiga, reforçando a proteção de infraestruturas críticas [4]. Dessa forma, a integração entre IA e operações de segurança fortalece a postura defensiva das organizações e viabiliza novas formas de colaboração entre humanos e máquinas diante dos desafios da segurança digital contemporânea [5].

Ao longo do tempo, diferentes abordagens foram empregadas para aprimorar a detecção e a mitigação de ameaças. Soluções iniciais baseavam-se em regras fixas, assinaturas ou políticas estáticas, como em antivírus tradicionais e *firewalls*. Embora importantes historicamente, esses mecanismos são limitados à detecção de ameaças já conhecidas e tendem a falhar diante de ataques novos ou sofisticados, especialmente os que exploram vulnerabilidades de dia zero. Para contornar essas limitações, técnicas de aprendizado de máquina passaram a ser utilizadas na identificação de padrões e no suporte à detecção de intrusões e anomalias em redes e sistemas [3]. Posteriormente, o avanço do aprendizado profundo ampliou essas possibilidades por meio do uso de redes neurais complexas, com aplicações em IDS/IPS e mecanismos de autenticação [2]. Mais recentemente, os modelos de linguagem de grande porte ganharam destaque por sua capacidade de processar e interpretar texto em contexto, o que abriu espaço para tarefas como análise de logs, interpretação de relatórios, triagem de alertas e simulação de ataques de *phishing* [4, 5].

Os LLMs representam um avanço relevante na cibersegurança porque conseguem operar em cenários que exigem raciocínio contextual, explicações detalhadas e correlação entre diferentes fontes textuais. Isso amplia seu uso para além da classificação simples, permitindo análises investigativas e apoio à tomada de decisão em ambientes marcados por incerteza. No entanto, a adoção desses modelos também traz desafios, como risco de interpretações equivocadas, excesso de confiança por parte dos usuários e custo computacional elevado, especialmente em SOCs, onde o volume diário de alertas é alto e a maioria deles corresponde a falsos positivos [4]. Essa situação, conhecida como *alert fatigue*, compromete a eficiência operacional e aumenta a chance de incidentes reais passarem despercebidos [5]. Nesse cenário, os LLMs podem atuar de forma estratégica,

auxiliando na triagem inteligente, na priorização de eventos relevantes e na geração de resumos explicativos que diminuam a sobrecarga das equipes de segurança.

A integração de LLMs com ferramentas de SOC já se mostrou promissora para reduzir carga de trabalho e melhorar a triagem de alertas. Estudos indicam que a combinação entre LLMs e plataformas de *Security Information and Event Management* (SIEM) pode elevar a precisão da detecção e oferecer suporte em linguagem natural às equipes de segurança [6, 7]. Também há evidências de que arquiteturas SIEM de próxima geração, quando associadas à IA, tendem a ganhar escalabilidade e eficiência [8, 9]. Apesar disso, persistem limitações práticas relacionadas ao custo de execução, à latência em cenários de tempo real, à privacidade dos dados e à dependência de serviços externos [7]. Em resposta a essas limitações, este trabalho propõe uma abordagem auto-hospedada, com modelos leves para filtragem de logs e modelos robustos combinados em *ensembles* para classificação, buscando equilibrar desempenho, privacidade e viabilidade operacional em um problema concreto de triagem de alertas de SOC.

3 Objetivos

Objetivo geral: desenvolver e avaliar um pipeline auto-hospedado em duas etapas, baseado em modelos de linguagem de grande porte, para triagem inteligente de alertas em SOCs, com foco na redução de falsos positivos, na preservação da privacidade dos dados e na viabilidade de implantação em ambientes reais.

Objetivos específicos:

1. projetar uma etapa inicial de filtragem de logs com LLMs leves, capaz de reduzir o volume de contexto enviado para classificação sem prejudicar a qualidade da decisão final;
2. implementar uma etapa de classificação com LLMs robustos auto-hospedados, adequada a ambientes isolados e sensíveis do ponto de vista de segurança da informação;
3. comparar estratégias de *ensemble* — votação por maioria, votação ponderada e seleção dinâmica — para identificar combinações mais estáveis e precisas na triagem de alertas;
4. avaliar a proposta em um ambiente controlado com geração de alertas reais e falsos positivos, medindo acurácia, precisão, *recall*, F1-score e tempo de inferência;
5. comparar o desempenho dos LLMs com modelos clássicos de aprendizado de máquina, evidenciando os ganhos práticos da abordagem proposta em operações de SOC.

4 Trabalhos relacionados

Diversos estudos têm investigado a aplicação de LLMs em diferentes domínios, inclusive em segurança e engenharia, com objetivos variados. O SecureLLM introduziu interfaces confidenciais capazes de compor modelos em tempo de execução a partir de silos de dados separados, garantindo controle de acesso e reduzindo risco de vazamento de informações sensíveis [10]. Bonner *et al.* exploraram o uso de LLMs para estabelecer rastreabilidade entre requisitos e artefatos de *Model-Based Systems Engineering* (MBSE), combinando *embeddings* semânticos e medidas de similaridade [11]. Já o estudo SecureFalcon propôs um *framework* escalável baseado em LLMs para detecção de vulnerabilidades de software [12]. Hassanin *et al.* apresentaram o PLLM-CS, um LLM pré-treinado voltado para segurança de redes satelitais, com foco em intrusões e atividades adversárias em comunicações críticas [13]. Em outra direção, Thomas *et al.* investigaram como LLMs podem apoiar moderadores humanos na identificação de conteúdo nocivo em plataformas digitais, propondo padrões colaborativos que melhoram a precisão e reduzem a carga cognitiva [14].

Em cenários mais específicos de cibersegurança, a literatura também destaca aplicações em inteligência de ameaças, testes ofensivos e apoio à decisão. Sufi propôs uma abordagem baseada em GPT para análise de relatórios históricos de incidentes, extraindo múltiplas dimensões de ataque, como atores, alvos e técnicas, com 96% de precisão, 98% de *recall* e 97% de F1-score em 214 incidentes avaliados [15]. Deng *et al.* apresentaram o PentestGPT, *framework* automatizado para testes de invasão que integra módulos de raciocínio, geração e análise, atingindo melhora de 228,6% na taxa de conclusão de tarefas em relação ao GPT-3.5 em ambientes HackTheBox e VulnHub [16]. Tariq *et al.* propuseram um modelo colaborativo entre humanos e IA que, em cenários de detecção de intrusão e classificação de imagens, elevou a taxa de detecção de 33,43% para 87,04% quando comparado a uma estratégia puramente automatizada [17].

No contexto operacional de SOC, as pesquisas se aproximam mais do problema tratado neste trabalho. Alekseichuk *et al.* combinaram modelos lógico-probabilísticos com especialistas virtuais baseados em LLMs para estimar parâmetros de ataques em redes corporativas, classificando caminhos de intrusão por probabilidade, tempo de execução e recursos exigidos [18]. Zangana *et al.* utilizaram GPT-4 e BERT para detecção de *phishing*, prevenção a fraudes e monitoramento de conformidade em instituições financeiras, relatando redução de 28% em incidentes de *phishing* e de 32% em falsos negativos para classificação de fraudes [19]. Singh *et al.* acompanharam, durante 10 meses, o uso de GPT-4 por analistas de SOC e verificaram que 93% das consultas estavam alinhadas às competências NICE, com apoio sobretudo em interpretação de comandos, revisão textual e contextualização [20]. Freitas *et al.* descreveram a arquitetura *Microsoft Copilot Guided Response*, implantada em escala industrial, com 87% de precisão e 41% de *recall*

na triagem, além de 99% de precisão e 62% de *recall* para recomendação de ações [21]. Nguyen *et al.* propuseram um pipeline em duas etapas para extração de técnicas MITRE ATT&CK em relatórios de *Cyber Threat Intelligence*, elevando o F1-score de cerca de 0,40 para valores acima de 0,90 em várias técnicas [22]. Outros autores também mostraram que a integração entre múltiplos LLMs e SIEMs reduz a carga de trabalho e melhora a correlação de alertas [6], enquanto copilotos baseados em RAG e integrados ao Wazuh podem reduzir falsos positivos e acelerar a resposta a incidentes [9]. Oniagbi *et al.* observaram precisão entre 70% e 80% na triagem de nível 1 com agentes baseados em LLM, ressaltando a necessidade de salvaguardas em casos de borda [7]. Marri *et al.* discutiram a integração de SIEMs de nova geração com *data lakes* e detecção de anomalias baseada em IA, enfatizando ganhos de escalabilidade e redução da fadiga de alertas [8].

A Tabela 1 resume parte desses trabalhos e ajuda a posicionar a presente proposta. Em comparação com a literatura, este estudo enfatiza uma solução auto-hospedada, com divisão explícita entre filtragem leve e classificação robusta, preservando a privacidade dos dados e buscando equilíbrio entre precisão, escalabilidade e implantação prática.

Tabela 1: Comparação entre abordagens recentes baseadas em LLMs para cibersegurança e o posicionamento da proposta desenvolvida neste trabalho.

Trabalho	Aplicação	Principais resultados
Alekseichuk <i>et al.</i> (2024) [18]	Simulação de caminhos de ataque em redes corporativas com especialistas probabilísticos baseados em LLMs.	Priorização de rotas de intrusão por probabilidade, tempo de execução e recursos necessários.
Zangana <i>et al.</i> (2025) [19]	Uso de GPT-4 e BERT para <i>phishing</i> , fraudes e conformidade no setor bancário.	Redução de 28% em incidentes de <i>phishing</i> e 32% menos falsos negativos em detecção de fraude.
Singh <i>et al.</i> (2025) [20]	Apoio a analistas de SOC com GPT-4 para interpretação de comandos e contextualização.	3.090 consultas de analistas; 93% alinhadas às competências NICE.
Freitas <i>et al.</i> (2025) [21]	Copilot corporativo para triagem e remediação em SOC.	87% de precisão e 41% de <i>recall</i> na triagem; 99% de precisão e 62% de <i>recall</i> na recomendação de ações.
Nguyen <i>et al.</i> (2025) [22]	Extração de técnicas MITRE ATT&CK em relatórios de CTI com GPT-3.5 + SciBERT.	F1-score elevado de cerca de 0,40 para valores superiores a 0,90 em várias técnicas.
Singh <i>et al.</i> (2024) [6]	Integração de múltiplos LLMs com SIEM para melhorar a correlação de alertas.	Melhoria na correlação de alertas e redução da carga operacional dos analistas.
Kurnia <i>et al.</i> (2025) [9]	Copiloto baseado em RAG integrado ao Wazuh.	Redução de falsos positivos e menor tempo de resposta em implantações reais.
Oniagbi <i>et al.</i> (2024) [7]	Avaliação de agentes baseados em LLMs para triagem de alertas de nível 1 em SOC.	Precisão entre 70% e 80%, dependendo da configuração e das restrições de latência.
Marri <i>et al.</i> (2024) [8]	SIEM de próxima geração com <i>data lakes</i> e IA para detecção de anomalias.	Ganhos de escalabilidade e redução da fadiga de alertas.
Este trabalho	Pipeline auto-hospedado com LLMs leves para filtragem e LLMs robustos para classificação de alertas em SOC.	F1-score de 0,90 e <i>recall</i> de 0,95, com preservação de privacidade, redução de ruído e viabilidade de implantação em infraestrutura própria.

5 Metodologia

5.1 Ambiente experimental e visão geral da arquitetura

O ambiente de testes foi projetado para emular a triagem de alertas em um SOC real, gerando alertas legítimos e falsos positivos com ferramentas abertas amplamente utilizadas em produção. A Figura 1 apresenta a visão geral do pipeline proposto. A arquitetura foi organizada em três grandes blocos inspirados em operações de SOC: coleta e correlação

dos eventos, orquestração e ingestão dos dados relevantes e, por fim, análise inteligente com LLMs. Na primeira etapa, os dispositivos geram logs que são enviados para o SIEM, responsável por correlacionar eventos e produzir alertas. Em seguida, um mecanismo de orquestração e automação recupera os logs relacionados a cada alerta e os encaminha para o pipeline de IA. Na etapa final, modelos leves executam a filtragem inicial do contexto e modelos robustos realizam a classificação do alerta.

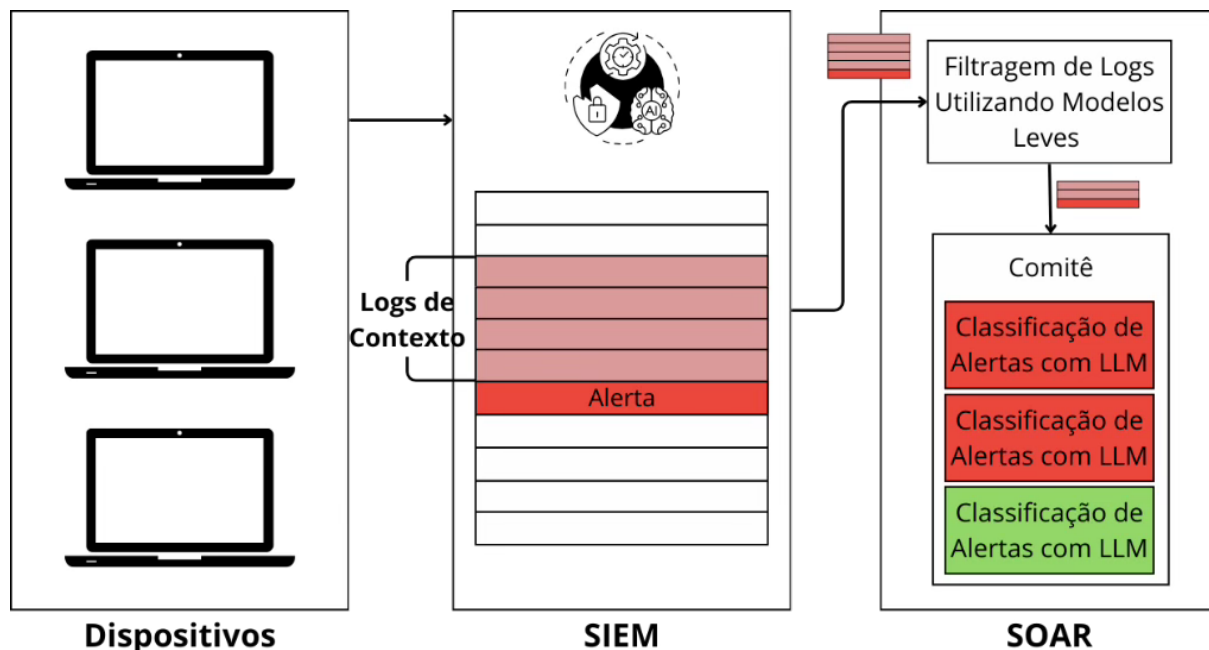


Figura 1: Visão geral do pipeline auto-hospedado proposto para triagem de alertas em SOC. A arquitetura integra coleta de logs, correlação em SIEM, orquestração por SOAR, filtragem com LLMs leves e classificação final com estratégias de *ensemble*.

Foram utilizadas duas máquinas com funções distintas. A primeira máquina consistiu em uma instância isolada do VirtualBox executando Windows 10 (64 bits), com 6 vCPU, 8 GB de RAM e 60 GB de armazenamento em disco, dedicada exclusivamente à emulação de ataques com o *Atomic Red Team*. A segunda máquina foi destinada à análise e ao processamento dos alertas gerados pelo SIEM, executando Ubuntu 22.04.5 LTS (64 bits) em um equipamento com processador AMD Ryzen Threadripper 7970X (32 núcleos/64 threads), 256 GB de RAM DDR4, GPU NVIDIA RTX 4090 e 4,1 TB de armazenamento NVMe SSD. Essa configuração forneceu capacidade computacional suficiente para processar grandes volumes de eventos e executar inferências com múltiplos LLMs, incluindo combinações em *ensemble*. Os scripts e configurações usados nos experimentos foram disponibilizados publicamente em [23].

Tabela 2: Resumo dos cenários de ataque utilizados para compor a base de dados e gerar alertas no SIEM.

Cenário 01		Cenário 02	
T1053.005	Scheduled Task Startup Script	T1047	WMI Execute Local Process
T1547.001	Registry Run Keys / Startup Folder	T1546.008	Accessibility Features
T1070.006	Timestomp a file with PowerShell	T1036.004	Create W32Time similar-named service
T1074.001	Local Data Staging	T1057	Process Discovery (tasklist)
T1105	Ingress Tool Transfer	T1082	System Information Discovery
T1087.001	Local Account Discovery	T1083	File and Directory Discovery (PowerShell)
T1552.004	Private Keys		
Cenário 03		Cenário 04	
T1053.005	Scheduled Task Startup Script	T1053.005	PowerShell Cmdlet Scheduled Task
T1047	WMI Execute Local Process	T1087.001	Enumerate all accounts via PowerShell
T1136.001	Create new Windows admin user	T1057	Process Discovery (tasklist)
T1136.002	Create new Windows domain admin user	T1083	File and Directory Discovery (PowerShell)
T1036.004	Create W32Time similar-named service	T1113	Screen Capture (CopyFromScreen)
T1140	Deobfuscate/Decode files or information		
T1552.001	Extract passwords with findstr		
Cenário 05		Cenário 06	
T1059.003	Create and execute batch script	T1059.001	Run BloodHound from local disk
T1543.003	Service installation (CMD)	T1543.003	Service installation (PowerShell)
T1564.001	Create hidden file (attrib)	T1562.001	Disable Windows Defender (DISM)
T1124	System Time Discovery (PowerShell)		
T1010	List Process Main Windows (C# .NET)		
T1056.001	Input Capture		
T1132.001	XOR-encoded data		
Cenário 07		Cenário 08	
T1047	WMI Execute with Encoded Command	T1562.004	Disable Microsoft Defender Firewall
T1552.001	WinPwn – sensitivefiles	T1003.001	Dump LSASS memory (ProcDump)
T1021.002	Map Admin Share (PowerShell)	T1082	Hostname Discovery (Windows)

5.2 Descrição da base de dados

A base de dados foi construída a partir de cenários simulados de ataque utilizando o framework *Atomic Red Team*, ferramenta amplamente empregada para emulação de técnicas ofensivas em ambientes controlados [24]. Essa abordagem permite simular comportamentos adversários segundo o framework MITRE ATT&CK [25], gerando eventos representativos para avaliar mecanismos de detecção em contexto de SOC. Além disso, foram adotados os cenários adversariais propostos em [26], complementados por cenários próprios alinhados aos objetivos desta pesquisa. Cada cenário foi planejado para gerar ao menos um alerta correspondente a cada TTP simulada. Após a execução dos ataques, os alertas resultantes foram misturados aleatoriamente com alertas falsos que representavam atividade legítima de usuário, reduzindo vieses e aproximando o experimento de uma situação operacional. Cada alerta foi associado a um conjunto de logs contextuais, permitindo uma análise mais rica pelos modelos.

A Tabela 2 apresenta as TTPs principais de cada cenário utilizado na geração da base. Ao todo, durante os testes finais, foram avaliados 60 alertas de SIEM, dos quais 41 correspondiam a ataques reais e 19 simulavam comportamentos benignos.

5.3 Pré-processamento e extração de eventos

Os experimentos foram realizados em Windows 10, em uma instância isolada do Virtual-Box [27], utilizando o Atomic Red Team para emular cenários realistas de ataque. A coleta dos eventos foi feita com o Sysmon [28], que registra atividades detalhadas do sistema operacional. Foi utilizada a configuração padrão do projeto aberto *Sysmon Modular* [29], que habilita um amplo conjunto de *Event IDs* e produz grande volume de alertas, inclusive de baixa relevância, para simular sobrecarga analítica. Os eventos coletados foram encaminhados para um SIEM baseado em Elasticsearch por meio do Winlogbeat [30, 31].

Para a análise com LLMs, foram extraídos todos os logs com carimbo de tempo dentro de uma janela de ± 1 minuto em relação a cada ataque, aumentando a quantidade de contexto disponível. Para reduzir ruído e concentrar a informação nos indicadores de comprometimento mais relevantes, foram preservados apenas os campos `process.name`, `process.command_line`, `file.path` e `winlog.task`. A ingestão foi automatizada por um script em Python, embora uma plataforma SOAR também pudesse cumprir essa função. A fim de evitar viés dos modelos em relação a nomes conhecidos de ferramentas, caminhos de arquivos ou identificadores facilmente reconhecíveis, os dados foram anonimizados. Cada nome sensível foi substituído por uma palavra aleatória única, mapeada de forma consistente por meio de uma tabela de correspondência. Em experimentos iniciais, hashes foram testados no lugar de palavras aleatórias, mas isso levou a aumento de falsos positivos, provavelmente porque os identificadores hash padronizavam excessivamente os padrões e eliminavam características distintivas.

A Figura 2 ilustra a cadeia de ferramentas empregada na geração da base e na análise dos eventos.

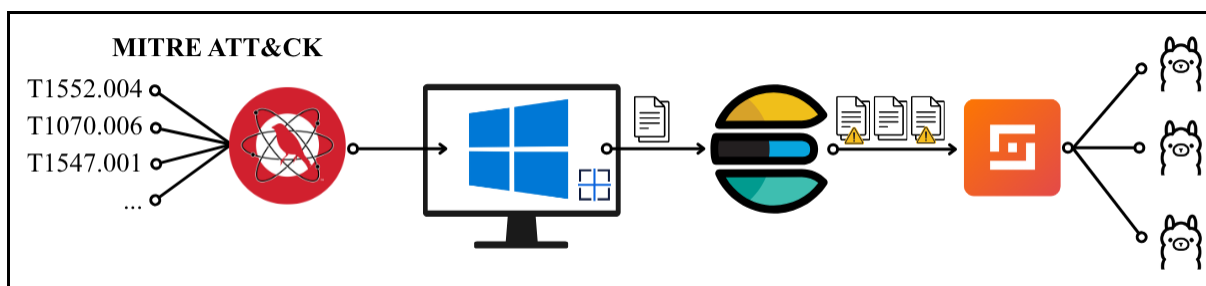


Figura 2: Ferramentas utilizadas na geração da base e na análise dos eventos. O ambiente inclui Atomic Red Team para emulação dos ataques, Sysmon para coleta de eventos, Winlogbeat para encaminhamento dos logs e Elasticsearch SIEM para agregação dos alertas em um ambiente Windows controlado via VirtualBox.

5.4 Filtragem de logs com modelos leves

Em um ambiente de SOC, o volume de logs é massivo e a maior parte dos registros não contribui diretamente para a identificação de incidentes. Por isso, foi implementado um

módulo dedicado de filtragem para mitigar a fadiga de alertas e entregar ao estágio seguinte um contexto mais limpo. Esse módulo emprega LLMs leves para classificar eventos como relevantes ou irrelevantes. Apenas as entradas consideradas importantes são encaminhadas aos modelos principais. Ao reduzir a quantidade de tokens de contexto, essa etapa preserva apenas a informação mais significativa para a análise posterior. Os modelos escolhidos para esse módulo foram selecionados com base no número de parâmetros, priorizando baixa latência de inferência:

- LLaMA 3.2 (3B);
- Phi4 Mini (3.8B);
- TinyLlama (1.1B);
- Gemma 3 (4B).

As saídas produzidas por essa etapa serviram de base para o módulo seguinte, responsável pela classificação aprofundada dos alertas.

5.5 Classificação de alertas com LLMs robustos

Foi implementada uma camada de classificação baseada em LLMs com o objetivo de reduzir falsos positivos e priorizar incidentes relevantes, diminuindo a carga de trabalho dos analistas. O LLM não foi concebido para substituir a tomada de decisão humana, mas para atuar como componente complementar dentro do pipeline, mantendo a criação de regras do SIEM e a validação final sob responsabilidade dos operadores. Devido à sensibilidade dos dados de log, foram adotados exclusivamente modelos auto-hospedados, viabilizando a implantação em rede isolada e impedindo exfiltração de dados. Por outro lado, esse tipo de implantação impõe restrições de hardware, já que a análise de grande quantidade de alertas diários demanda infraestrutura robusta para manter tempos de inferência aceitáveis. Os modelos avaliados nessa etapa foram:

- DeepSeek-R1 (14B);
- Phi-4 (14B);
- Mistral-Nemo (12B);
- LLaMA 3.1 (8B);
- Gemma 3 (12B);
- Qwen3 (14B).

A seleção considerou dois critérios principais: a faixa de parâmetros, entre 8 e 14 bilhões, e a relevância dos modelos no ecossistema de LLMs. Essa faixa buscou equilibrar capacidade de generalização e viabilidade de implantação auto-hospedada, enquanto a popularidade dos modelos refletiu adoção comunitária e suporte contínuo.

5.6 Estratégias de *ensemble*

Como os modelos auto-hospedados permitem a execução paralela de múltiplas inferências sobre os mesmos alertas, foram avaliadas estratégias de *ensemble* para verificar ganhos de desempenho. Três métodos foram adotados, conforme a Figura 3. O primeiro, votação por maioria, emprega três modelos distintos para classificar cada alerta como *Interesting* ou *Not Interesting*, escolhendo como decisão final a classe selecionada pela maioria. O segundo, votação ponderada, combina as previsões de acordo com os escores de confiança retornados por cada inferência, adotando como decisão final a classe com maior confiança acumulada. Por fim, a seleção dinâmica utiliza a inferência do modelo com maior confiança como classificação final.

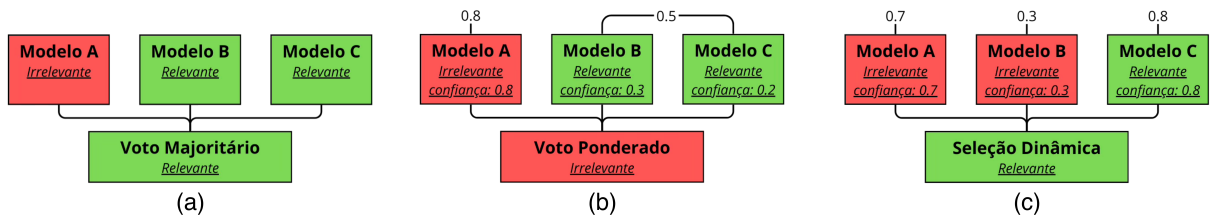


Figura 3: Estratégias de *ensemble* adotadas no pipeline proposto: (a) votação por maioria, (b) votação ponderada e (c) seleção dinâmica para combinar as saídas de múltiplos LLMs auto-hospedados.

5.7 Métricas de avaliação

O desempenho dos modelos integrados ao SIEM foi avaliado com métricas capazes de equilibrar precisão e cobertura da detecção, além de considerar a viabilidade operacional em termos de tempo de inferência. Antes de definir as fórmulas, considerou-se a terminologia clássica: verdadeiro positivo (TP), falso positivo (FP), falso negativo (FN) e verdadeiro negativo (TN). A acurácia foi calculada por:

$$\text{Acurcia} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (1)$$

que representa a proporção global de classificações corretas. A precisão foi definida por:

$$\text{Preciso} = \frac{TP}{TP + FP}, \quad (2)$$

indicando a proporção de alertas classificados como relevantes que de fato correspondem a incidentes reais. O *recall* foi calculado por:

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (3)$$

e mede a capacidade do modelo de capturar incidentes reais presentes no conjunto de teste. O F1-score, utilizado para equilibrar precisão e *recall*, foi obtido por:

$$\text{F1-score} = \frac{2 \times \text{Preciso} \times \text{Recall}}{\text{Preciso} + \text{Recall}}. \quad (4)$$

Além disso, o tempo de inferência foi calculado como a média do tempo real de execução de cada modelo durante os testes práticos, uma métrica essencial para avaliar a viabilidade da solução em ambientes com alto volume de alertas.

5.8 Modelos de aprendizado de máquina de referência

Além dos experimentos com LLMs, também foram avaliados modelos tradicionais de aprendizado de máquina como linha de base. O objetivo foi verificar como algoritmos clássicos se comportam no mesmo cenário de triagem de alertas considerado para os LLMs. Foram selecionados quatro modelos consolidados em classificação binária e detecção de anomalias: *Decision Tree*, *Random Forest*, *Logistic Regression* e *Linear SVC*. O conjunto de treinamento foi gerado no mesmo ambiente controlado utilizado para os demais experimentos, com coleta de logs em uma máquina virtual Windows durante a execução de ataques via Atomic Red Team. Os ataques empregados posteriormente na avaliação dos LLMs foram excluídos do treinamento, garantindo teste em dados não vistos e comparabilidade entre as abordagens.

Após a coleta com Sysmon, os dados foram organizados e limpos para remoção de redundâncias e padronização da estrutura das entradas. Campos textuais, como linhas de comando e nomes de arquivos, foram convertidos em representações numéricas para viabilizar o treinamento. Em seguida, a base foi dividida em subconjuntos de treino e teste mantendo equilíbrio entre alertas reais e falsos positivos. Esses modelos forneceram uma referência importante para analisar as vantagens dos LLMs em precisão, escalabilidade e robustez no contexto do SOC.

6 Resultados e discussão

Durante os testes, os 60 alertas avaliados foram acompanhados por linhas de log contextuais coletadas em uma janela de ± 1 minuto em torno do carimbo temporal de cada evento. Para lidar com os limites de contexto durante a inferência, os modelos principais processa-

ram os alertas em blocos predefinidos de até 1.500 tokens por inferência. Consequentemente, um único alerta poderia gerar múltiplas inferências, dependendo da quantidade de dados contextuais associados. Ao final, uma estratégia de votação por maioria foi aplicada para determinar a classificação consolidada de cada alerta.

6.1 Desempenho da filtragem de logs

A Figura 4 apresenta a porcentagem de redução do contexto obtida por cada modelo leve. Nessa etapa, a porcentagem de filtragem não representa diretamente a taxa de descarte correto de eventos irrelevantes, pois a base contém milhares de linhas de log e a verdade de terreno foi definida no nível do alerta, não da linha individual. Assim, o indicador reflete a redução global de contexto após a filtragem. A relevância prática desse corte é inferida a partir do impacto na etapa seguinte: se um modelo remove grande parcela do contexto e ainda assim mantém ou melhora métricas finais como acurácia, precisão, *recall* e F1-score, então ele se mostra eficiente para a filtragem.

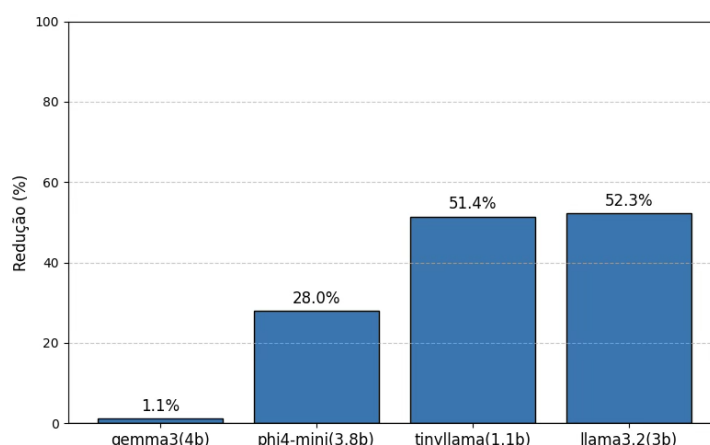


Figura 4: Percentual de redução do contexto de logs obtido por cada LLM leve utilizado na etapa de filtragem.

Os resultados indicaram que o LLaMA 3.2 (3B) foi o modelo mais eficaz para essa função, reduzindo o volume de contexto em mais da metade sem comprometer o desempenho da classificação final. Além disso, conforme apresentado na Tabela 3, esse modelo também obteve o menor tempo médio de inferência entre os avaliados, com 0,051 s por execução, reforçando sua adequação para a etapa de filtragem. Modelos como Phi-4 Mini e Gemma 3 também apresentaram ganhos em relação ao uso dos dados brutos, especialmente em precisão, embora com tempos de resposta substancialmente maiores. Já o TinyLlama foi o único a apresentar queda em acurácia e F1-score, além de um tempo de inferência superior ao do LLaMA 3.2, sugerindo que uma filtragem excessivamente agressiva ou mal calibrada pode prejudicar a decisão final.

A Figura 5 compara o impacto do contexto bruto e do contexto filtrado por cada

Tabela 3: Comparação do tempo de inferência entre os LLMs leves empregados na etapa de filtragem de logs.

Modelo	Parâmetros (B)	Tempo (s)
LLaMA 3.2	3.0	0.051
TinyLlama	1.1	0.353
Phi-4 Mini	3.8	0.810
Gemma 3	4.0	0.866

modelo leve nas métricas finais dos modelos robustos. Observa-se que o uso de filtragem adequada tende a melhorar a consistência do estágio seguinte, reduzindo ruído e concentrando a análise em evidências mais relevantes.

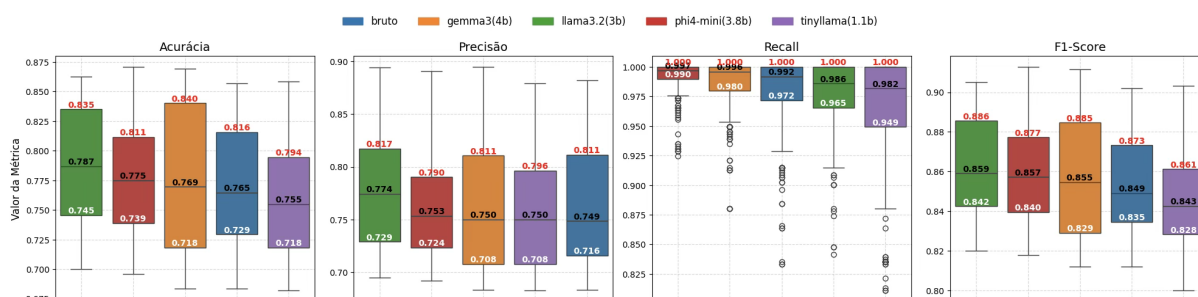


Figura 5: Avaliação comparativa de acurácia, precisão, *recall* e F1-score considerando o contexto bruto e o contexto filtrado pelos modelos leves.

6.2 Desempenho da classificação com LLMs robustos

A Figura 6 compara os modelos robustos avaliados em termos de acurácia, precisão, *recall* e F1-score. O modelo DeepSeek-R1 (no_think) apresentou o melhor equilíbrio geral, atingindo a maior mediana de F1-score ($\approx 0,903$) e desempenho consistente em acurácia e *recall*. O Qwen3 (no_think) se destacou em precisão ($\approx 0,882$), enquanto Gemma 3 e LLaMA 3.1 alcançaram valores de *recall* próximos de 1,0, identificando eficientemente incidentes relevantes. Contudo, esses mesmos modelos apresentaram queda no F1-score em razão do desequilíbrio entre precisão e *recall*, indicando tendência maior a falsos positivos. A comparação entre modos *think* e *no think* mostrou que cadeias de raciocínio mais longas introduziram vieses e pioraram o desempenho quando comparadas às configurações sem raciocínio expandido.

A Tabela 4 apresenta os tempos médios de inferência dos principais modelos robustos. Os dados mostram que os modelos configurados em modo *think* apresentam latência significativamente superior às versões *no think*. No caso do Qwen3, a diferença foi de aproximadamente 7 segundos; para o DeepSeek-R1, de cerca de 3,7 segundos. Esses resultados sugerem que cadeias de raciocínio mais longas não apenas degradam a qualidade da inferência nesse cenário, como também elevam de forma substancial a latência do processo.

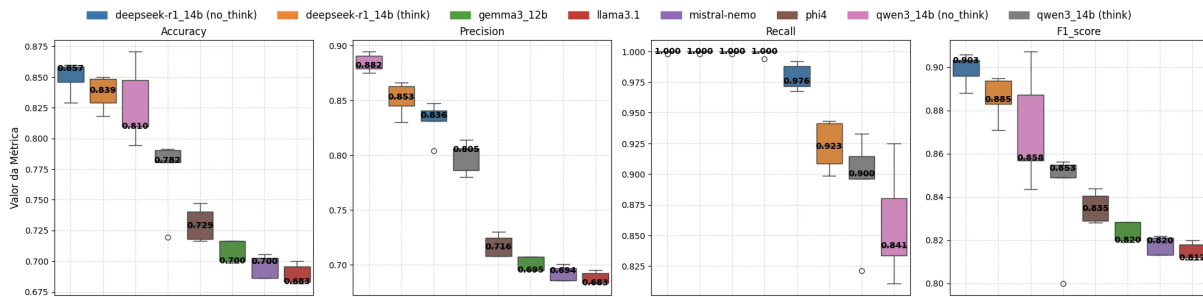


Figura 6: Comparação entre os LLMs robustos avaliados no estágio de classificação em termos de acurácia, precisão, *recall* e F1-score.

Tabela 4: Comparação do tempo de inferência entre os LLMs robustos empregados na etapa de classificação.

Modelo	Parâmetros (B)	Tempo (s)
LLaMA 3.1	8.0	1.223
Mistral-Nemo	12.0	1.496
Gemma 3	12.0	1.937
Qwen3 (No Think)	14.0	1.938
Phi-4	14.0	2.331
DeepSeek-R1 (No Think)	14.0	3.733
DeepSeek-R1 (Think)	14.0	7.477
Qwen3 (Think)	14.0	8.973

6.3 Efetividade das estratégias de *ensemble*

A avaliação das diferentes estratégias de *ensemble* mostrou ganhos significativos em relação ao desempenho individual dos modelos. Todas as combinações possíveis entre os modelos robustos foram testadas, e os resultados médios por estratégia são mostrados na Figura 7. Entre as estratégias analisadas, a seleção dinâmica foi a mais eficaz em termos médios, exibindo desempenho consistente e equilibrado em todas as métricas. Isso ocorre porque a seleção dinâmica explora as forças de cada modelo em cenários específicos, reduzindo o impacto de classificações equivocadas e produzindo decisão final mais estável.

A votação por maioria, embora mais simples, também trouxe ganhos em relação aos modelos individuais, demonstrando que mesmo uma agregação básica já contribui para maior robustez na triagem. Em contraste, os modelos individuais exibiram maior variabilidade de resultados, com métricas menos consistentes. A votação por maioria foi aplicada a combinações de três modelos, enquanto votação ponderada e seleção dinâmica foram usadas tanto em configurações com dois quanto com três modelos. Apesar de a seleção dinâmica ter atingido o melhor desempenho médio, os melhores resultados absolutos foram obtidos por maioria e votação ponderada. Nessas configurações, ambas

alcançaram resultados praticamente idênticos ao combinar DeepSeek-R1 em seus dois modos de execução com Qwen3 em modo *no think*, conforme mostra a Figura 8.

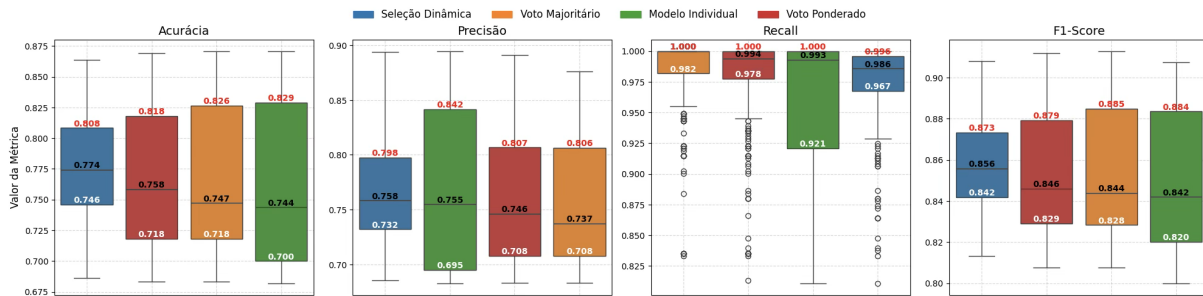


Figura 7: Comparação entre as estratégias de *ensemble* — votação por maioria, votação ponderada e seleção dinâmica — em termos de acurácia, precisão, *recall* e F1-score.

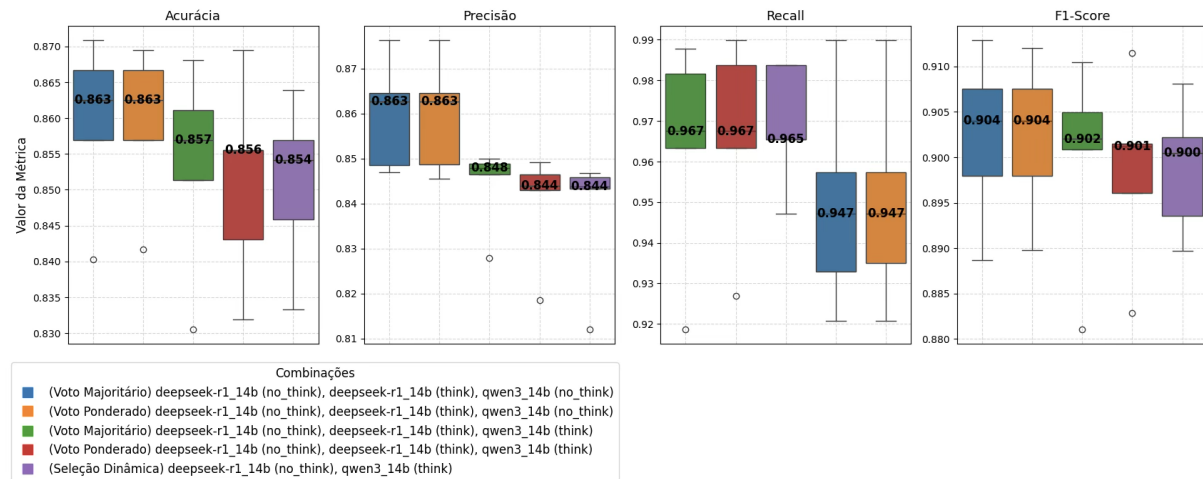


Figura 8: Cinco melhores configurações de *ensemble* entre os LLMs robustos avaliados.

6.4 Discussão comparativa

A Tabela 5 resume o desempenho dos modelos tradicionais de aprendizado de máquina. Os resultados correspondem às médias obtidas a partir de diferentes estratégias de filtragem aplicadas à base descrita anteriormente. Entre os algoritmos testados, o *Decision Tree* atingiu o maior F1-score (0,82), seguido por *Logistic Regression* (0,80), *Random Forest* (0,80) e *Linear SVC* (0,79). Embora esses resultados mostrem que métodos clássicos podem desempenhar razoavelmente bem em classificação binária, sua precisão e acurácia permaneceram abaixo dos valores alcançados pelos LLMs auto-hospedados.

Tabela 5: Desempenho dos classificadores tradicionais de aprendizado de máquina utilizados como linha de base.

Modelo	Acurácia	Precisão	Recall	F1-score
Decision Tree	0.70	0.71	0.96	0.82
Logistic Regression	0.68	0.69	0.96	0.80
Random Forest	0.67	0.69	0.95	0.80
Linear SVC	0.66	0.68	0.95	0.79

Embora o *recall* desses modelos tenha permanecido alto ($\approx 0,95$ – $0,96$), a precisão foi substancialmente menor ($\approx 0,68$ – $0,71$), revelando forte tendência a falsos positivos. Esse comportamento pode ser atribuído à capacidade limitada desses modelos para interpretar contexto e sequencialidade presentes nos logs. Além disso, o treinamento exigiu esforço adicional em preparação dos dados, extração de atributos e ajuste de hiperparâmetros. Em contraste, os LLMs analisaram os mesmos alertas sem necessidade de treinamento supervisionado adicional, alcançando F1-scores acima de 0,90 com melhor equilíbrio entre precisão e *recall*. Isso sugere que, para dados textuais ricos e não estruturados, os LLMs são mais adequados do que abordagens clássicas puramente vetoriais.

Quando comparados aos estudos revisados na Subseção 2.1, os resultados deste trabalho se mostram alinhados ou superiores às tendências reportadas na literatura, ainda que os conjuntos de dados e as configurações experimentais sejam diferentes. Oniagbi *et al.* [7], por exemplo, reportaram precisão entre 70% e 80% na triagem de nível 1 com agentes baseados em LLMs, enquanto Freitas *et al.* [21] alcançaram 87% de precisão e 41% de *recall* em um copiloto corporativo de larga escala. Nos experimentos auto-hospedados desta pesquisa, os *ensembles* mantiveram simultaneamente alta precisão e alto *recall* (0,90 e 0,95, respectivamente), indicando equilíbrio competitivo entre cobertura e redução de falsos positivos sob uma configuração totalmente privada. De forma semelhante, Nguyen *et al.* [22] alcançaram F1-score em torno de 0,90 em tarefas controladas de extração de CTI com GPT-3.5 + SciBERT, enquanto a proposta aqui desenvolvida atingiu desempenho comparável em triagem de alertas em tempo real no contexto de SOC. Esses padrões reforçam que a abordagem apresentada oferece efetividade compatível com sistemas proprietários de grande escala, ao mesmo tempo em que melhora privacidade, escalabilidade e viabilidade de implantação.

7 Conclusão

Este trabalho apresentou um pipeline auto-hospedado em duas etapas para aprimorar a triagem de alertas em Centros de Operações de Segurança por meio de modelos de linguagem de grande porte. A arquitetura proposta combinou LLMs leves para filtragem de logs e redução de ruído com LLMs robustos organizados em estratégias de *ensem-*

ble, incluindo votação por maioria, votação ponderada e seleção dinâmica, buscando alta acurácia e estabilidade operacional. Os experimentos, conduzidos em um ambiente controlado de Windows com simulações do Atomic Red Team e cenários baseados no MITRE ATT&CK, demonstraram que os *ensembles* alcançaram mediana de F1-score de 0,90 e *recall* de 0,95, superando tanto modelos individuais quanto classificadores tradicionais de aprendizado de máquina.

Entre os métodos avaliados, a votação por maioria apresentou o melhor equilíbrio entre precisão e *recall*, enquanto a seleção dinâmica exibiu a menor variação de desempenho entre combinações. Além disso, a filtragem leve com LLaMA 3.2 (3B) reduziu em mais da metade o número de logs irrelevantes processados na segunda etapa, sem comprometer a qualidade da classificação final, o que confirma a eficiência da estratégia em duas etapas. Do ponto de vista prático, a solução proposta enfrenta um problema concreto dos SOC's modernos: o excesso de alertas e a dificuldade de análise em tempo hábil. Ao manter o processamento em infraestrutura própria, a proposta também reduz dependência de serviços externos e preserva a confidencialidade dos dados de segurança.

Assim, conclui-se que arquiteturas auto-hospedadas baseadas em LLMs podem oferecer desempenho compatível com soluções corporativas em larga escala, ao mesmo tempo em que preservam privacidade, escalabilidade e autonomia operacional. Os resultados finais obtidos indicam que a combinação entre filtragem leve de contexto e classificação robusta por *ensemble* constitui uma alternativa viável para apoiar analistas de segurança em operações reais de triagem de alertas.

Referências Bibliográficas

- [1] M. Baruwat Chhetri, S. Tariq, R. Singh, F. Jalalvand, C. Paris, and S. Nepal, "Towards human-ai teaming to mitigate alert fatigue in security operations centres," *ACM Transactions on Internet Technology*, vol. 24, no. 3, pp. 1–22, 2024.
- [2] I. Hasanov, S. Virtanen, A. Hakkala, and J. Isoaho, "Application of large language models in cybersecurity: A systematic literature review," *IEEE Access*, 2024.
- [3] M. Vielberth, F. Böhm, I. Fichtinger, and G. Pernul, "Security operations center: A systematic study and open challenges," *IEEE Access*, vol. 8, pp. 227 756–227 779, 2020.
- [4] S. Tariq, R. Singh, M. B. Chhetri, S. Nepal, and C. Paris, "Bridging expertise gaps: The role of llms in human-ai collaboration for cybersecurity," *arXiv preprint arXiv:2505.03179*, 2025.

- [5] S. Tariq, M. Baruwat Chhetri, S. Nepal, and C. Paris, "Alert fatigue in security operations centres: Research challenges and opportunities," *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–38, 2025.
- [6] Y. Singh, N. Patel, and S. K. Shandilya, "Enhancing security operations center efficiency through multi-model integration of large language models and siem systems," 2024.
- [7] O. Oniagbi, A. Hakkala, and I. Hasanov, "Evaluation of llm agents for the soc tier 1 analyst triage process," Ph.D. dissertation, MS thesis, University of Turku, 2024.
- [8] R. Marri, S. Varanasi, and S. V. K. Chaitanya, "Integrating next-generation siem with data lakes and ai: Advancing threat detection and response," *Journal of Artificial Intelligence General science (JAIGS) ISSN: 3006-4023*, vol. 3, no. 1, pp. 446–465, 2024.
- [9] R. Kurnia, F. Widyatama, I. M. Wibawa, Z. A. Brata, G. A. Nelistiani, H. Kim *et al.*, "Enhancing security operations center: Wazuh security event response with retrieval-augmented-generation-driven copilot," *Sensors (Basel, Switzerland)*, vol. 25, no. 3, p. 870, 2025.
- [10] A. Alabdulkareem, C. Arnold, Y. Lee, P. M. Feenstra, B. Katz, and A. Barbu, "Securellm: New private and confidential interfaces with llms," 2025.
- [11] M. Bonner, M. Zeller, G. Schulz, and A. Savu, "Llm-based approach to automatically establish traceability between requirements and mbse," in *INCOSE International Symposium*, vol. 34, no. 1. Wiley Online Library, 2024, pp. 2542–2560.
- [12] M. A. Ferrag, A. Battah, N. Tihanyi, R. Jain, D. Maimuț, F. Alwahedi, T. Lestable, N. S. Thandi, A. Mechri, M. Debbah *et al.*, "Securefalcon: Are we there yet in automated software vulnerability detection with llms?" *IEEE Transactions on Software Engineering*, 2025.
- [13] M. Hassanin, M. Keshk, S. Salim, M. Alsubaie, and D. Sharma, "Pllm-cs: Pre-trained large language model (llm) for cyber threat detection in satellite networks," *Ad Hoc Networks*, vol. 166, p. 103645, 2025.
- [14] K. Thomas, P. G. Kelley, D. Tao, S. Meiklejohn, O. Vallis, S. Tan, B. Bratanič, F. T. Ferreira, V. K. Eranti, and E. Bursztein, "Supporting human raters with the detection of harmful content using large language models," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 2772–2789.
- [15] F. Sufi, "An innovative gpt-based open-source intelligence using historical cyber incident reports," *Natural Language Processing Journal*, vol. 7, p. 100074, 2024.

- [16] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "{PentestGPT}: Evaluating and harnessing large language models for automated penetration testing," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 847–864.
- [17] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, "A2c: A modular multi-stage collaborative decision framework for human–ai teams," *Expert Systems with Applications*, vol. 282, p. 127318, 2025.
- [18] L. Alekseichuk, D. Lande, and O. Novikov, "Application of large language models for assessing parameters and possible scenarios of cyberattacks on information and communication systems," *Theoretical and Applied Cybersecurity*, vol. 6, no. 1, 2024.
- [19] H. M. Zangana, H. S. Mohammed, and M. M. Husain, "The role of large language models in enhancing cybersecurity measures: Empirical evidence from regional banking institutions," *SISTEMASI*, vol. 14, no. 5, pp. 2018–2027, 2025.
- [20] R. Singh, S. Tariq, F. Jalalvand, M. B. Chhetri, S. Nepal, C. Paris, and M. Lochner, "Llms in the soc: An empirical study of human-ai collaboration in security operations centres," *arXiv preprint arXiv:2508.18947*, 2025.
- [21] S. Freitas, J. Kalajdjieski, A. Gharib, and R. McCann, "Ai-driven guided response for security operation centers with microsoft copilot for security," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 191–200.
- [22] H. Cuong Nguyen, S. Tariq, M. Baruwat Chhetri, and B. Quoc Vo, "Towards effective identification of attack techniques in cyber threat intelligence reports using large language models," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 942–946.
- [23] L. R. Ferreira, "Test code for alert emulation in socs with llms," GitHub repository, 2025, accessed: Aug. 21, 2025. [Online]. Available: <https://github.com/Fiddelis/AI-Security-Labs>
- [24] Red Canary, "Atomic red team: Repository of adversary emulation tests," GitHub repository, 2025, accessed: Aug. 21, 2025. [Online]. Available: <https://github.com/redcanaryco/atomic-red-team/>
- [25] MITRE Corporation, "MITRE ATT&CK®: Adversarial Tactics, Techniques, and Common Knowledge," Website, 2025, accessed: Aug. 21, 2025. [Online]. Available: <https://attack.mitre.org/>
- [26] J. Elgh, "Comparison of adversary emulation tools for reproducing behavior in cyber attacks," 2022.

- [27] Oracle Corporation, “Virtualbox,” Software, 2025, accessed: Aug. 21, 2025. [Online]. Available: <https://www.virtualbox.org/>
- [28] Microsoft Corporation, “Sysmon – system monitor,” Software, 2025, accessed: Aug. 21, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- [29] O. Hartong, “sysmon-modular: A repository of sysmon configuration modules,” GitHub repository, 2025, accessed: 16 July 2025. [Online]. Available: <https://github.com/olafhartong/sysmon-modular>
- [30] Elastic, “Elasticsearch: Search and analytics engine,” Software, 2025, accessed: Aug. 21, 2025. [Online]. Available: <https://www.elastic.co/elasticsearch>
- [31] —, “Winlogbeat: Lightweight shipper for windows event logs,” Software, 2025, accessed: Aug. 21, 2025. [Online]. Available: <https://www.elastic.co/beats/winlogbeat>